Atividade em grupo | Cap 1 - A Aurora Estabelece os Primeiros Sistemas da Colônia

Integrantes
RM 575105 | Kelven Carlos Luiz de Souza
RM 575601 | Aldo Roschel
RM 575921 | Larissa Bressanini Lehn
RM 575981 | Nicole Luiza Mascarenhas Zerbato

Abstract

Este relatório apresenta um sistema em Python para monitorar o balanço energético de uma colônia fictícia, estimar a geração de energia solar e eólica a partir de registros históricos e emitir recomendações operacionais por meio de regras condicionais. Os dados de geração e de condições ambientais são organizados em listas aninhadas, identificadas por fonte energética. A estimativa utiliza um modelo de regressão linear por mínimos quadrados, calculado individualmente para cada fonte a partir de três observações históricas. Em seguida, o sistema soma as estimativas de geração à energia atualmente armazenada, subtrai o consumo informado e utiliza o saldo energético projetado para selecionar uma ação. O relatório descreve a organização dos dados, o procedimento matemático de ajuste das retas, as regras implementadas e as limitações da solução. Os gráficos apresentados representam os pontos históricos fornecidos no código e as respectivas retas ajustadas.

1 Introdução

O gerenciamento de energia é relevante para a continuidade das atividades de uma colônia, especialmente quando sua geração depende de fontes variáveis, como luz solar e vento.

O objetivo deste trabalho é desenvolver um sistema simples, em Python, que reúna dados históricos de geração e de condições ambientais, estime a geração solar e eólica por meio de regressão linear por mínimos quadrados e produza uma recomendação operacional com base em regras condicionais.

A implementação procura atender aos requisitos da atividade: organizar informações em estruturas de dados, representar fontes energéticas, aplicar decisões automáticas, realizar uma previsão simples e comparar geração, consumo e energia disponível.

2 Escopo e limitações deste relatório técnico

Este relatório documenta a implementação apresentada no repositório público `https://github.com/kelvencarlos/fiap-03` e o código-fonte reproduzido no Apêndice A.

O escopo contempla:

- armazenamento de três observações históricas de geração e de condições ambientais para as fontes solar e eólica;
- cálculo de uma relação linear individual para cada fonte por meio do método dos mínimos quadrados;

- utilização das observações históricas disponíveis no cálculo dos coeficientes;
- soma das duas estimativas para obter a geração total prevista;
- cálculo do saldo energético projetado, considerando a energia atual, o consumo informado e a geração prevista;
- seleção de uma ação por meio de estruturas condicionais.

A implementação é didática e utiliza valores de exemplo. Não há, no código apresentado, conexão com sensores, armazenamento persistente, coleta meteorológica em tempo real ou controle físico dos equipamentos. As unidades das entradas ambientais e da energia não são especificadas no programa; por isso, os valores são tratados neste relatório como unidades numéricas de entrada e de energia.

3 Estimativa preditiva

3.1 Organização dos dados históricos

Os dados são organizados em duas listas principais. A lista `historicoEnergiaGerada` associa o identificador de cada fonte à sequência de energia gerada. A lista `historicoEntradas` associa o mesmo identificador à sequência correspondente de condições ambientais.

Tabela 1: Dados históricos utilizados no exemplo.

Fonte	**Observação 1**	**Observação 2**	**Observação 3**
Solar – entrada (luz)	9	11	13
Solar – energia gerada	30	35	40
Eólica – entrada (vento)	8	10	12
Eólica – energia gerada	20	25	30

As listas internas agrupam o nome da fonte e seus valores, permitindo acessar separadamente o histórico solar e o eólico por índice. A organização mantém os registros de entrada e de geração em estruturas paralelas. Assim, a correspondência entre cada entrada ambiental e sua geração depende de que as posições das duas listas permaneçam alinhadas.

3.2 Modelo de regressão linear

Para estimar a geração de energia, o programa utiliza a regressão linear por mínimos quadrados. Diferentemente de um cálculo baseado somente em dois registros, esse método considera todas as observações disponíveis para encontrar a reta que melhor representa a relação entre a variável de entrada e a energia gerada.

O modelo linear é representado por:

$$\hat{y} = ax + b, \tag{1}$$

em que $\hat{y}$ representa a geração estimada, x representa a condição ambiental informada, a é o coeficiente angular e b é o intercepto.

O método dos mínimos quadrados determina os coeficientes da reta minimizando a soma dos quadrados das diferenças entre os valores observados e os valores estimados:

$$S(a, b) = \sum_{i=1}^{n} \left(y_i - (ax_i + b)\right)^2 . \tag{2}$$

Nessa expressão, $S(a, b)$ representa a soma dos quadrados dos resíduos, isto é, das diferenças entre os valores reais e os estimados pelo modelo. O termo y_i representa o valor observado na posição i, enquanto $ax_i + b$ corresponde ao valor estimado pela reta para a mesma observação. O índice i identifica cada registro, e n representa a quantidade total de observações.

A diferença entre o valor observado e o estimado é chamada de resíduo. Ela pode ser representada por:

$$e_i = y_i - \hat{y}_i. \tag{3}$$

Como os resíduos podem ser positivos ou negativos, elevá-los ao quadrado evita que diferenças de sinais opostos se anulem durante a soma. Além disso, essa operação atribui maior peso às diferenças de maior magnitude.

Assim, o objetivo do método é encontrar os valores de a e b que produzem o menor valor possível para $S(a, b)$. Para isso, são utilizadas as seguintes expressões:

$$a = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}, \tag{4}$$

$$b = \frac{\sum y - a\sum x}{n}. \tag{5}$$

Nessas expressões, n representa a quantidade de observações; $\sum x$ corresponde à soma das entradas; $\sum y$ corresponde à soma das gerações observadas; $\sum xy$ representa a soma dos produtos entre cada entrada e sua respectiva geração; e $\sum x^2$ representa a soma dos quadrados das entradas.

As fórmulas permitem calcular diretamente os coeficientes da reta que minimiza a soma dos quadrados dos resíduos, sem a necessidade de testar manualmente diferentes combinações de valores para a e b.

3.3 Cálculo do modelo solar

Para a fonte solar, os dados históricos são:

Entrada (x)	**Geração** (y)	**Produto** (xy)
9	30	270
11	35	385
13	40	520

A partir desses valores, são obtidas as seguintes somas:

$$\begin{aligned} n &= 3 \\ \sum x &= 9 + 11 + 13 = 33 \\ \sum y &= 30 + 35 + 40 = 105 \\ \sum xy &= 270 + 385 + 520 = 1175 \\ \sum x^2 &= 9^2 + 11^2 + 13^2 = 371 \end{aligned}$$

Substituindo os valores na Equação 4, obtém-se o coeficiente angular:

$$\begin{aligned} a &= \frac{3 \cdot 1175 - (33 \cdot 105)}{3 \cdot 371 - 33^2} \\ &= \frac{3525 - 3465}{1113 - 1089} \\ &= \frac{60}{24} \\ &= 2{,}5. \end{aligned}$$

Em seguida, o intercepto é calculado pela Equação 5:

$$\begin{aligned} b &= \frac{105 - (2{,}5 \cdot 33)}{3} \\ &= \frac{105 - 82{,}5}{3} \\ &= \frac{22{,}5}{3} \\ &= 7{,}5. \end{aligned}$$

Portanto, o modelo linear para a geração solar é:

$$\hat{y}_{solar} = 2{,}5x + 7{,}5. \tag{6}$$

3.4 Cálculo do modelo eólico

Para a fonte eólica, os dados históricos são:

Entrada (x)	**Geração** (y)	**Produto** (xy)
8	20	160
10	25	250
12	30	360

As somas correspondentes são:

$$\begin{aligned} n &= 3 \\ \sum x &= 8 + 10 + 12 = 30 \\ \sum y &= 20 + 25 + 30 = 75 \\ \sum xy &= 160 + 250 + 360 = 770 \\ \sum x^2 &= 8^2 + 10^2 + 12^2 = 308 \end{aligned}$$

Substituindo os valores na Equação 4:

$$\begin{aligned} a &= \frac{3 \cdot 770 - (30 \cdot 75)}{3 \cdot 308 - 30^2} \\ &= \frac{2310 - 2250}{924 - 900} \\ &= \frac{60}{24} \\ &= 2{,}5. \end{aligned}$$

O intercepto é:

$$\begin{aligned} b &= \frac{75 - (2{,}5 \cdot 30)}{3} \\ &= \frac{75 - 75}{3} \\ &= 0. \end{aligned}$$

Portanto, o modelo linear para a geração eólica é:

$$\hat{y}_{eolica} = 2{,}5x. \tag{7}$$

3.5 Estimativa da geração total de energia

A geração total prevista é calculada pela soma das estimativas individuais:

$$\hat{G}_{total} = \hat{y}_{solar} + \hat{y}_{eolica}. \tag{8}$$

Substituindo as equações obtidas:

$$\hat{G}_{total} = (2{,}5x_{solar} + 7{,}5) + (2{,}5x_{eolica}). \tag{9}$$

O programa recebe separadamente os valores de entrada solar e eólica, calcula as duas estimativas e retorna a soma como previsão total de geração.

3.6 Interpretação do ajuste

Os três pontos históricos de cada fonte estão perfeitamente alinhados. Por esse motivo, a reta ajustada pelo método dos mínimos quadrados coincide com a reta que seria obtida utilizando apenas dois pontos.

Essa coincidência é uma característica específica dos dados utilizados no exemplo, e não uma equivalência entre os métodos. Em conjuntos de dados nos quais as observações não estejam perfeitamente alinhadas, o método dos mínimos quadrados considera todos os pontos e encontra a reta que minimiza a soma dos quadrados dos resíduos.

3.7 Pontos históricos e modelos lineares

A Figura 1 apresenta os registros solares e a reta ajustada pelo método dos mínimos quadrados. A Figura 2 apresenta os registros eólicos e sua respectiva reta. Os pontos representam os valores históricos fornecidos no código; as linhas representam as equações calculadas.

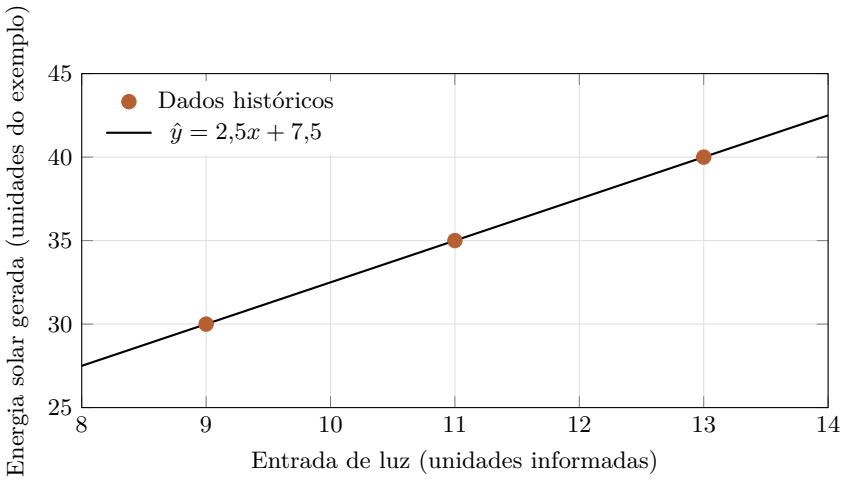


Figura 1: Dados históricos e modelo de regressão linear para geração solar.

4 Funcionalidades

O sistema implementa três capacidades principais.

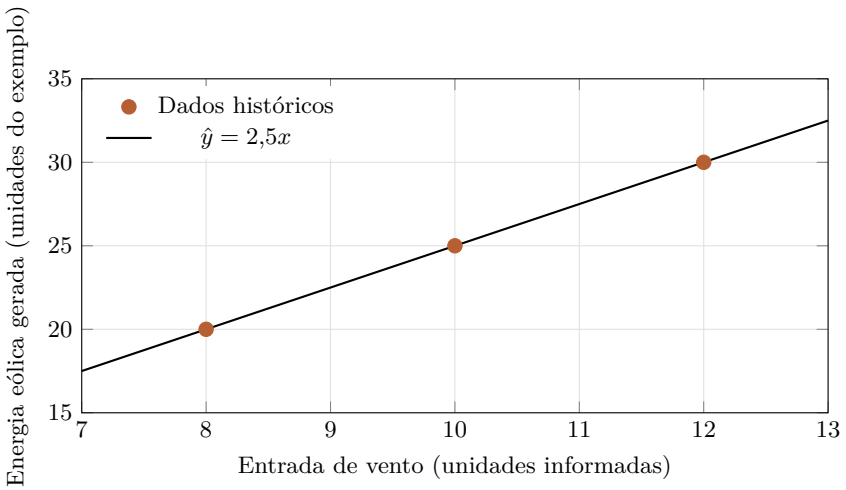


Figura 2: Dados históricos e modelo de regressão linear para geração eólica.

4.1 Estimativa da geração de energia

A função `preverEnergiaGerada` calcula separadamente a estimativa solar e a eólica e retorna a soma. As funções auxiliares `calcularCoeficienteAngular` e `calcularIntercepto` concentram os cálculos matemáticos da regressão linear por mínimos quadrados.

A função de cálculo do coeficiente angular utiliza as somas das entradas, das gerações, dos produtos entre entradas e gerações e dos quadrados das entradas. Dessa forma, as três observações históricas são consideradas no ajuste da reta.

4.2 Monitoramento do balanço energético

A função `calcularSaldoEnergiaColonia` reúne a geração prevista pelas fontes solar e eólica, a reserva atual de energia e o consumo informado. Diferentemente da versão anterior, a geração prevista passa a ser adicionada à energia atual antes da subtração do consumo.

O saldo energético projetado é calculado por:

$$S_{final} = E_{atual} - C + \hat{G}_{total}. \tag{10}$$

Em que:

- S_{final} representa o saldo energético projetado após considerar o consumo e a geração estimada;
- E_{atual} representa a reserva de energia informada pelo usuário;
- C representa o consumo atual informado;
- $\hat{G}_{total}$ representa a soma das estimativas de geração solar e eólica.

O resultado é uma projeção baseada nos valores fornecidos, e não uma medição efetiva da energia que será gerada. A função também exibe a previsão total de geração, o consumo informado, a reserva atual e o saldo energético projetado.

4.3 Regras de decisão

Após calcular o saldo energético projetado, o programa avalia as condições na ordem em que aparecem no código:

1. Se o saldo energético projetado for menor que o consumo informado, o sistema recomenda manter o suporte à vida ligado e desligar sistemas não essenciais.

2. Caso contrário, se o saldo energético projetado for maior que o consumo e o consumo for maior que a geração prevista, recomenda reduzir o consumo de energia.
3. Nos demais casos, recomenda armazenar a energia excedente.

A primeira condição tem prioridade sobre as seguintes. Por utilizar `if`, `elif` e `else`, apenas uma das mensagens de ação é emitida em cada execução.

As comparações utilizam o saldo energético projetado, e não somente a reserva atual. Dessa maneira, as regras consideram a geração estimada no cálculo da disponibilidade energética.

5 Avaliação e resultados

A avaliação apresentada nesta versão é analítica e utiliza exclusivamente os dados de exemplo já definidos no código. Não foram fornecidos registros de execução com valores de entrada específicos nem uma série de testes; portanto, não são atribuídas métricas de acurácia ou desempenho ao sistema.

Com a aplicação do método dos mínimos quadrados às três observações de cada fonte, as equações calculadas são:

- Solar: $\hat{y}_{solar} = 2{,}5x + 7{,}5$;
- Eólica: $\hat{y}_{eolica} = 2{,}5x$.

Nos últimos pontos históricos, as entradas são 13 para luz e 12 para vento. As respectivas estimativas são 40 e 30 unidades de energia, totalizando 70 unidades. Esse total corresponde à soma das gerações históricas nos últimos pontos e serve como conferência dos cálculos, não como uma medição futura independente.

Como os três pontos de cada conjunto estão perfeitamente alinhados, os valores calculados pelos mínimos quadrados coincidem com os obtidos pelo procedimento anterior, baseado nos dois últimos pontos. Entretanto, a nova implementação utiliza todas as observações no cálculo dos coeficientes, tornando o procedimento matematicamente adequado à regressão linear por mínimos quadrados.

O saldo energético projetado depende dos valores digitados pelo usuário para luz, vento, consumo e energia atual. A geração prevista é somada à reserva atual e, em seguida, o consumo é subtraído. Como esses valores de entrada não foram especificados no relatório, não é apresentado um exemplo de saída como se tivesse sido efetivamente executado.

6 Conclusão

O trabalho apresenta um sistema integrado em Python que organiza históricos de geração solar e eólica, calcula estimativas por meio de regressão linear por mínimos quadrados e utiliza regras condicionais para emitir recomendações sobre o uso de energia. A solução reúne estruturas de dados, funções, operações matemáticas e lógica de decisão em um único fluxo de execução.

A atualização da implementação permite que as observações históricas sejam utilizadas no cálculo dos coeficientes da reta, em vez de considerar apenas os dois registros mais recentes. Nos dados utilizados, os resultados permanecem iguais porque os pontos de cada fonte estão perfeitamente alinhados. Ainda assim, a nova abordagem representa uma aplicação mais completa do método de regressão linear por mínimos quadrados.

Outra alteração importante foi a inclusão da geração prevista no cálculo do saldo energético. Dessa forma, o programa passou a considerar a reserva atual, o consumo informado e a geração estimada para produzir um saldo projetado que serve de base para as regras condicionais.

A implementação demonstra a aplicação dos conceitos solicitados na atividade, mas ainda utiliza dados fixos e um modelo de previsão simplificado. Para evoluir a solução, seria necessário ampliar o histórico, validar as entradas, avaliar o modelo com dados não utilizados

no cálculo e comparar a geração prevista com a geração efetivamente observada. Também seria importante testar diferentes cenários para verificar se as regras preservam a prioridade dos sistemas essenciais.

Repositório

O código-fonte do projeto está disponível publicamente em:

https://github.com/kelvencarlos/fiap-03

O repositório deve ser consultado para acessar a versão entregue do sistema e seu README com instruções e exemplos de execução.

Referências

OPENAI. *GPT-4 Technical Report*. 2023. Disponível em: https://arxiv.org/abs/2303.08774. Acesso em: 10 nov. 2026.

FIAP. *Atividade integradora: sistema inteligente da colônia*. Material didático da disciplina. 2026.

A Código-fonte em Python

O código abaixo apresenta a implementação atualizada, que utiliza o método dos mínimos quadrados para calcular os coeficientes das retas de geração solar e eólica. As estimativas individuais são somadas e utilizadas no cálculo do saldo energético projetado.

```

historicoEnergiaGerada = [
    ["energia_solar", [30, 35, 40]],
    ["energia_eolica", [20, 25, 30]],
]

historicoEntradas = [
    ["energia_solar", [9, 11, 13]],
    ["energia_eolica", [8, 10, 12]]
]


def calcularCoeficienteAngular(historicoEnergiasGerada, historicoEntradas):

    quantidadeObservacoes = len(historicoEnergiasGerada)

    somaEntradas = (
        historicoEntradas[0]
        + historicoEntradas[1]
        + historicoEntradas[2]
    )

    somaGeracoes = (
        historicoEnergiasGerada[0]
        + historicoEnergiasGerada[1]
        + historicoEnergiasGerada[2]
    )

    somaMultiplicacoesEntradasGeracoes = (
        historicoEntradas[0] * historicoEnergiasGerada[0]
        + historicoEntradas[1] * historicoEnergiasGerada[1]
        + historicoEntradas[2] * historicoEnergiasGerada[2]
    )

    somaQuadradosEntradas = (
```

```
        historicoEntradas[0] * historicoEntradas[0]
        + historicoEntradas[1] * historicoEntradas[1]
        + historicoEntradas[2] * historicoEntradas[2]
    )

    coeficienteAngular = (
        (quantidadeObservacoes * somaMultiplicacoesEntradasGeracoes)
        - (somaEntradas * somaGeracoes)
    ) / (
        (quantidadeObservacoes * somaQuadradosEntradas)
        - (somaEntradas * somaEntradas)
    )

    return coeficienteAngular


def calcularIntercepto(
    coeficienteAngular,
    historicoEnergiasGeradas,
    historicoEntradas
):

    somaEnergiasGeradas = (
        historicoEnergiasGeradas[0]
        + historicoEnergiasGeradas[1]
        + historicoEnergiasGeradas[2]
    )

    somaEntradas = (
        historicoEntradas[0]
        + historicoEntradas[1]
        + historicoEntradas[2]
    )

    quantidadeObservacoes = len(historicoEnergiasGeradas)

    intercepto = (
        somaEnergiasGeradas - (coeficienteAngular * somaEntradas)
    ) / quantidadeObservacoes

    return intercepto


def preverEnergiaGerada(
    entradaEnergia,
    historicoEnergiaGerada,
    historicoEntradas
):

    coeficienteAngularEnergia = calcularCoeficienteAngular(
        historicoEnergiaGerada[1],
        historicoEntradas[1]
    )

    intercepto = calcularIntercepto(
        coeficienteAngularEnergia,
        historicoEnergiaGerada[1],
        historicoEntradas[1]
    )

    energiaGerada = (
        coeficienteAngularEnergia * entradaEnergia + intercepto
    )

```

```
    if historicoEnergiaGerada[0] == "energia_solar":
        print(
            "Coeficiente angular: {}\n"
            "Entrada de energia: {}\n"
            "Intercepto: {}\n"
            "Equação da reta: y = {} x + {}\n"
            "Energia gerada: {} (Solar)\n".format(
                coeficienteAngularEnergia,
                entradaEnergia,
                intercepto,
                coeficienteAngularEnergia,
                intercepto,
                energiaGerada,
            )
        )
    else:
        print(
            "Coeficiente angular: {}\n"
            "Entrada de energia: {}\n"
            "Intercepto: {}\n"
            "Equação da reta: y = {} x + {}\n"
            "Energia gerada: {} (Eólica)\n".format(
                coeficienteAngularEnergia,
                entradaEnergia,
                intercepto,
                coeficienteAngularEnergia,
                intercepto,
                energiaGerada,
            )
        )

    return energiaGerada


def monitorarEnergiaIndividualmente(
    entradaEnergia,
    historicoEnergiaGerada,
    historicoEntradas
):

    previsaoEnergiaGerada = preverEnergiaGerada(
        entradaEnergia,
        historicoEnergiaGerada,
        historicoEntradas
    )

    return previsaoEnergiaGerada


entradaEnergiaSolar = float(input("Insira a quantidade de luz: "))

energiaSolarGerada = monitorarEnergiaIndividualmente(
    entradaEnergiaSolar,
    historicoEnergiaGerada[0],
    historicoEntradas[0]
)

entradaEnergiaEolica = float(input("Insira a quantidade de vento: "))

energiaEolicaGerada = monitorarEnergiaIndividualmente(
    entradaEnergiaEolica,
    historicoEnergiaGerada[1],
    historicoEntradas[1]
)
```

```
164
165 energiaAtualColonia = float(
166     input("Insira a reserva de energia atual da colônia: ")
167 )
168
169 consumoEnergia = float(
170     input("Insira o consumo de energia atual da colônia: ")
171 )
172
173
174 def calcularSaldoEnergiaColonia(
175     energiaAtualColonia,
176     energiaSolarGerada,
177     energiaEolicaGerada,
178     consumoEnergia
179 ):
180
181     energiaPrevista = energiaSolarGerada + energiaEolicaGerada
182
183     saldoEnergiaFinal = (
184         energiaAtualColonia - consumoEnergia + energiaPrevista
185     )
186
187     print(
188         "\nEnergia gerada: {}\n"
189         "Consumo atual da colônia: {}\n"
190         "Reserva de energia atual da colônia: {}\n"
191         "Saldo final de energia da colônia: {}\n".format(
192             energiaPrevista,
193             consumoEnergia,
194             energiaAtualColonia,
195             saldoEnergiaFinal
196         )
197     )
198
199     if saldoEnergiaFinal < consumoEnergia:
200         print(
201             "Ação: Manter suporte à vida ligado "
202             "e desligar sistemas não essenciais"
203         )
204
205     elif (
206         saldoEnergiaFinal > consumoEnergia
207         and consumoEnergia > energiaPrevista
208     ):
209         print("Ação: Reduzir consumo de energia")
210
211     else:
212         print("Ação: Armazenar energia excedente")
213
214
215 calcularSaldoEnergiaColonia(
216     energiaAtualColonia,
217     energiaSolarGerada,
218     energiaEolicaGerada,
219     consumoEnergia
220 )
```